PROJECT DOCUMENTATION

02

Technical Specification

Architecture, data model, build pipeline, authentication and the AirShare signalling protocol as implemented on 31 October 2026.

PREPARED FOR
Rishav Prasad
Owner and maintainer

SITE
rishavprasad.in
Cloudflare Workers

VERSION
1.0

DATE
31 October 2026

Contents

Technical Specification · version 1.0 · 14 sections

01 Scope and design goals 3

02 Technology stack 3

03 System architecture 4

04 Build and delivery pipeline 5

05 Content model 5

06 Data model 6

07 HTTP API 7

08 Authentication and sessions 8

09 AirShare protocol 9

10 Security design 10

11 Performance and caching 11

12 SEO, accessibility and browsers 11

13 Known limitations and technical debt 12

14 Extending the system 13

01

Scope and design goals

The portfolio is a static-first personal site with a small, deliberately bounded serverless backend. This document describes what exists in the repository, not an aspirational design.

Goals

- Serve every public page as pre-rendered HTML from Cloudflare's edge, so the portfolio stays fast and available even when the API is unavailable.
- Keep all content editable by the owner without a code deploy, while keeping the public site static.
- Accept contact messages without exposing the owner's inbox address to scrapers; publish the address as a deliberate alternative.
- Provide a LAN-only, PIN-gated file transfer tool where file bytes never reach the server.
- Stay inside free tiers: Cloudflare Workers, Workers Static Assets, D1 and Turnstile; no paid database, object storage or relay.

Non-goals and explicit boundaries

- No server-side rendering at request time and no separate origin server.
- No user accounts, comments, analytics dashboard or e-commerce.
- No public API for writing content; the only writer is the authenticated admin.
- No file relay, TURN server or cloud storage for AirShare transfers.
- No reliance on Cloudflare Pages; deployment is a Worker with Static Assets.

02

Technology stack

LAYER	TECHNOLOGY	VERSION IN LOCKFILE	ROLE
Framework	Next.js (App Router, static export)	16.4.3	Routing, pre-rendering and build output in `out/`.
UI runtime	React / React DOM	19.3.1	Hydration for theme, menu, contact form, AirShare and admin.
Language	TypeScript (strict)	7.0.2	Application and Worker code; separate tsconfig for the Worker.
Styling	Tailwind CSS	4.3.3	Utility classes plus CSS custom properties for the palette.
Icons	lucide-react	1.65.0	Interface and content icons, mapped from a ten-key allow-list.
API framework	Hono	4.13.14	Worker routes, middleware, cookies and error handling.
Compute	Cloudflare Workers	compatibility date 2026-10-01	Serves the API and static assets at the edge.

LAYER	TECHNOLOGY	VERSION IN LOCKFILE	ROLE
Database	Cloudflare D1 (SQLite)	database `rishav-portfolio`	Content document, messages, media index and AirShare signalling.
Object storage	Cloudflare R2	optional, not enabled	Would back `/media/*`; currently unbound.
Bot protection	Cloudflare Turnstile	managed service	Contact form verification.
Tooling	Playwright, Vitest, Wrangler	1.64.1 / 5.0.5 / 4.156.0	Browser tests, unit tests and deployment.

Runtime dependencies are limited to five packages; everything else is build-time tooling.

03

System architecture

Cloudflare serves static files without invoking the Worker. Only `/api/*` and `/media/*` are routed through it. The admin dashboard is itself a static page that calls the API from the browser.

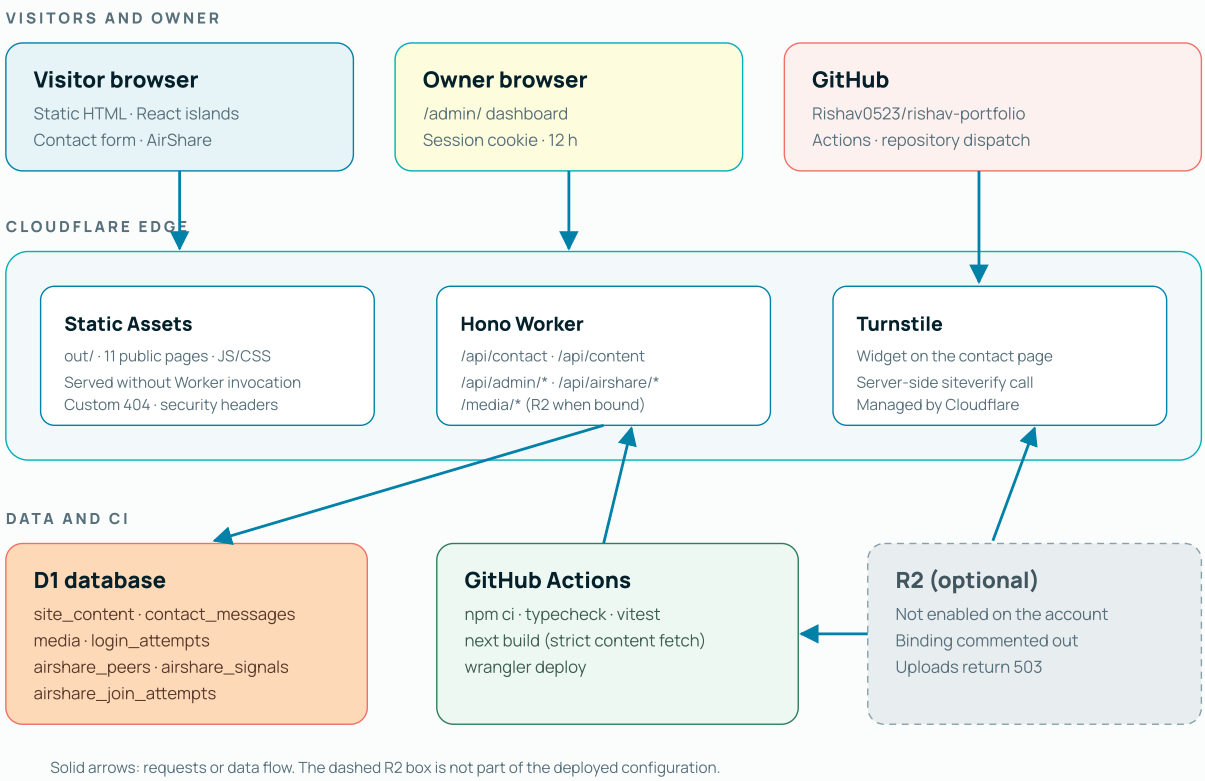


Figure 1. Deployed topology. Static requests never enter the Worker; the API and media paths do.

Request paths

1. **Public page:** browser requests `/projects/` → Static Assets returns `out/projects/index.html` with the site headers. No database call is made at request time.
2. **Contact:** browser posts JSON to `/api/contact` → Worker validates, verifies Turnstile, enforces the IP hash limit, inserts into D1, returns 201.
3. **Admin edit:** browser sends the whole content document to `PUT /api/admin/content` with the session cookie → Worker validates against the schema and upserts row 1 of `site_content`.
4. **Publish:** `POST /api/admin/publish` → Worker calls GitHub's dispatch endpoint → Actions rebuilds → the build's `prebuild` step fetches `/api/content` → `wrangler deploy` replaces the static bundle.
5. **AirShare:** both browsers post presence and WebRTC signalling to `/api/airshare/*`; once connected, file chunks flow directly over a WebRTC data channel.

04

Build and delivery pipeline

1. **Prebuild content snapshot.** `scripts/fetch-content.mjs` fetches `/api/content` when `SITE_URL` or `CONTENT_API_URL` is set, validates the shape, and rewrites `content/published.json`. In CI, `CONTENT_FETCH_STRICT=1` turns any failure into a failed build instead of deploying stale content.
2. **Static export.** `next build` pre-renders every route with `output: "export"` and `trailingSlash: true` into `out/`. Images are unoptimized because there is no Next image server.
3. **Checks.** CI runs `npm ci`, `npm run typecheck` (app and Worker) and `npm test` (Vitest) before building.
4. **Deploy.** `npx wrangler deploy` uploads `out/` as Static Assets and the Worker bundle. The custom domain route is declared in `wrangler.jsonc`.

The committed `content/published.json` is both a build input and the offline fallback. Local builds without `SITE_URL` keep the snapshot; this is why the site can be rebuilt without reaching production.

> **Deployment scope.** The workflow deploys Workers and assets. D1 migrations are applied manually with `wrangler d1 migrations apply`; the deploy token is scoped to Workers. The GitHub repository uses secrets `CLOUDFLARE_API_TOKEN` and `CLOUDFLARE_ACCOUNT_ID`.

05

Content model

All editable content is one versioned JSON document with a fixed shape. The same TypeScript types in `lib/content.ts` are used by the pages, the admin editor and the Worker validator.

Top-level fields

FIELD	TYPE	ITEMS ALLOWED	PER-FIELD LIMITS
`version`	number	must equal 1	—
`socials`	array	6	label allow-list; href 500
`education`	array	10	degree 160, institution 160, notes 6 × 240
`skillGroups`	array	20	items 20 × 60; icon allow-list
`domains`	string[]	30	80 characters each
`leadership`	array	20	points 12 × 300
`journey`	array	30	year 10, title 120, detail 300
`projects`	array	50	detail 1200; links 6
`achievements`	array	50	detail 600; category allow-list
`awards`	array	100	result 40
`certifications`	array	100	href optional
`resumeSummary`	string	—	2000 characters
`resumePdf`	object	—	href plus `approvedForPublish`
`profileFacts`	array	12	label 40, value 120

Validation rules

- All required strings are trimmed and must be non-empty where the type requires a value.
- Hrefs must start with `/` or match `http(s)://`; this blocks `javascript:` and `data:` links from content.
- Project slugs use `[a-z0-9-]+` and must be unique within the document.
- Icons must be one of ten keys: code, globe, database, shield, brain, rocket, terminal, megaphone, lightbulb, newspaper.
- Project status is one of “In development”, “Completed”, “Shipped”; achievement category is “Leadership” or “Event organising”.
- Serialized documents larger than 512 KB are rejected. Validation reports at most 25 errors.

The public pages do not fall back to defaults on a malformed D1 row: they render the committed snapshot when the database is unavailable, and the admin editor refuses to load invalid content at all. This prevents a bad read from becoming a destructive write.

06

Data model

Seven application tables live in one D1 database. D1 also keeps its own `d1_migrations` table and an internal `_cf_KV` table.

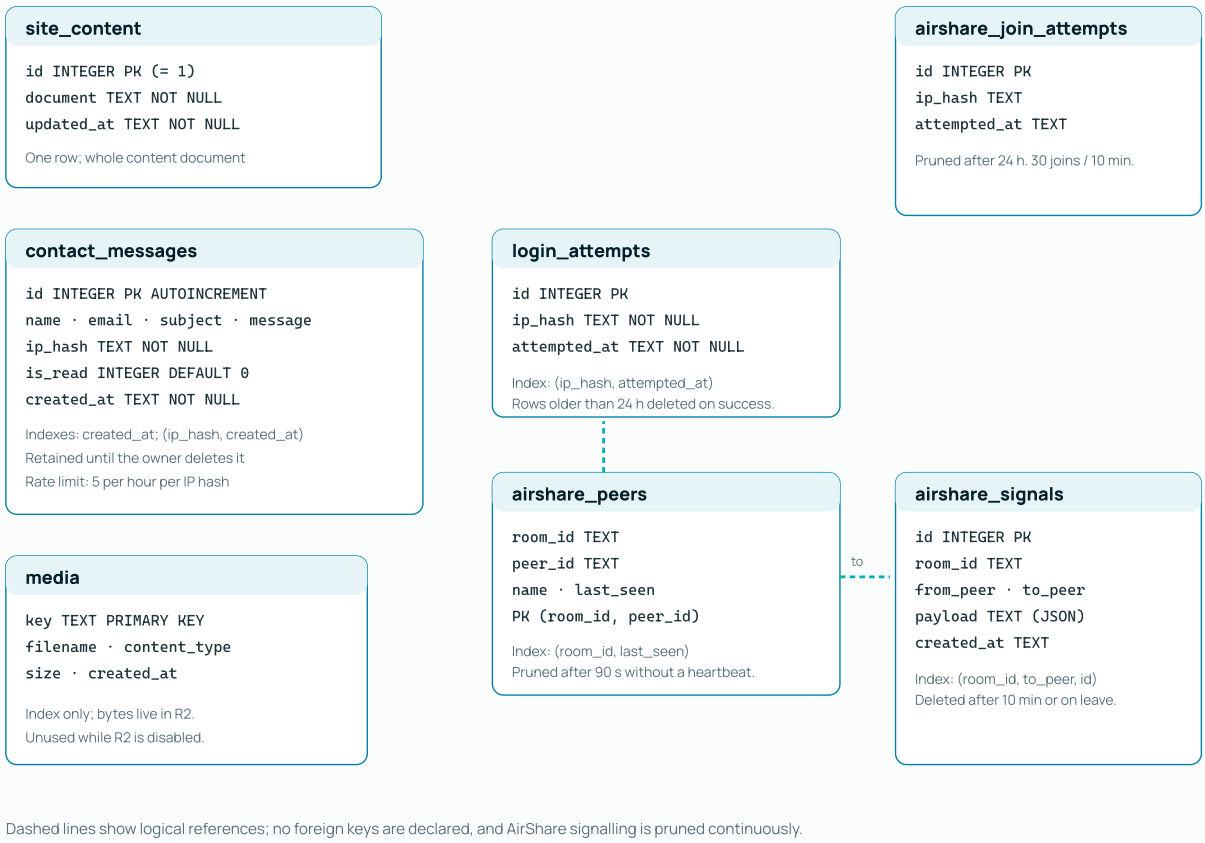


Figure 2. Entity relationships as defined by `migrations/0001_init.sql` and `0002_airshare.sql`.

Table notes

- `site_content` is constrained to a single row (`CHECK (id = 1)`), so there is no history table. Backups are the history mechanism.
- `contact_messages` stores a SHA-256 hash prefixed with `contact:` rather than an IP address.
- `media` would index uploaded images; the object bytes would live in R2 under `media/<uuid>.<ext>`.
- AirShare peer and signal rows are operational, not archival. They are deleted by time-based pruning and on leave.

07

HTTP API

The Worker exposes three groups: public endpoints, the session-protected admin group, and the AirShare signalling group. Every response is JSON; the full contract is in `API_DOCUMENTATION.md`.

METHOD AND PATH	AUTH	PURPOSE
`GET /api/health`	None	Reports service and database reachability. 200 or 503.
`GET /api/content`	None	Current content document; `max-age=60`-like caching.
`POST /api/contact`	Turnstile	Store a message; 201 on success, 429 when rate limited.
`POST /api/admin/login`	Password	Set the 12-hour session cookie.
`POST /api/admin/logout`	Session	Delete the cookie.
`GET /api/admin/session`	Session	Session probe used by the dashboard.
`GET/PUT /api/admin/content`	Session	Read or replace the content document.
`POST /api/admin/publish`	Session	Trigger the GitHub rebuild dispatch.
`GET/PATCH/DELETE /api/admin/messages`	Session	List (limit 100), mark read, delete.
`GET/POST/DELETE /api/admin/media`	Session	R2-backed image library; 503 while unbound.
`POST /api/admin/resume`	Session	R2-backed PDF replacement; 503 while unbound.
`POST /api/airshare/join`	PIN	Create or join a room; returns room id and peers.
`POST /api/airshare/heartbeat`	Room and peer	Refresh presence and return the peer list.
`POST /api/airshare/signal`	Room and peer	Queue one SDP or ICE payload for another peer.
`GET /api/airshare/signals`	Room and peer	Poll up to 50 signals after a cursor.
`POST /api/airshare/leave`	Room and peer	Remove presence and related signals.

Error behaviour

Errors use `{"error": "message"}` with a specific status: 400 for validation, 401 for missing or invalid session, 403 for a disallowed origin, 404 for unknown resources, 409 for a full AirShare room, 410 for an expired room membership, 429 for rate limits and 503 for missing configuration or dependency failure. Unexpected exceptions are logged server-side and returned as a generic 503 “Something went wrong.”

08

Authentication and sessions

Password verification

The stored secret is a string in the form `pbkdf2_sha256$100000$<salt>$<hash>`. The Worker decodes the salt and expected hash, derives bits with PBKDF2-HMAC-SHA256 at exactly 100,000 iterations, and compares the byte arrays in constant time. Any other iteration count is rejected before derivation, because Workers throw rather than return false above 100,000.

Session token

A successful login creates a payload `{sub:"admin", iat, exp}`, base64url-encodes it, signs it with HMAC-SHA256 using `SESSION_SECRET`, and stores `body.signature` in the cookie `admin_session`. The cookie is HttpOnly, SameSite=Strict, path `/`, and Secure when the request is HTTPS. Verification recomputes the signature, compares in constant time, checks the subject and expiry, and rejects anything else.

Defences

- Eight failed logins per 15 minutes per IP hash; stale attempts are cleaned up after a successful login.
- Login and all mutating admin requests check the `Origin` header against the request origin and `SITE_URL`; requests without an Origin (for example, server-to-server) are allowed.
- The dashboard sends the content document only after the session probe succeeds; a 401 on save or publish returns to the login screen.
- The admin page is noindex and disallowed in robots.txt, but those are not security controls.

09

AirShare protocol

AirShare is a thin signalling layer over a WebRTC data channel. D1 carries presence and SDP/ICE messages; it never sees file bytes.

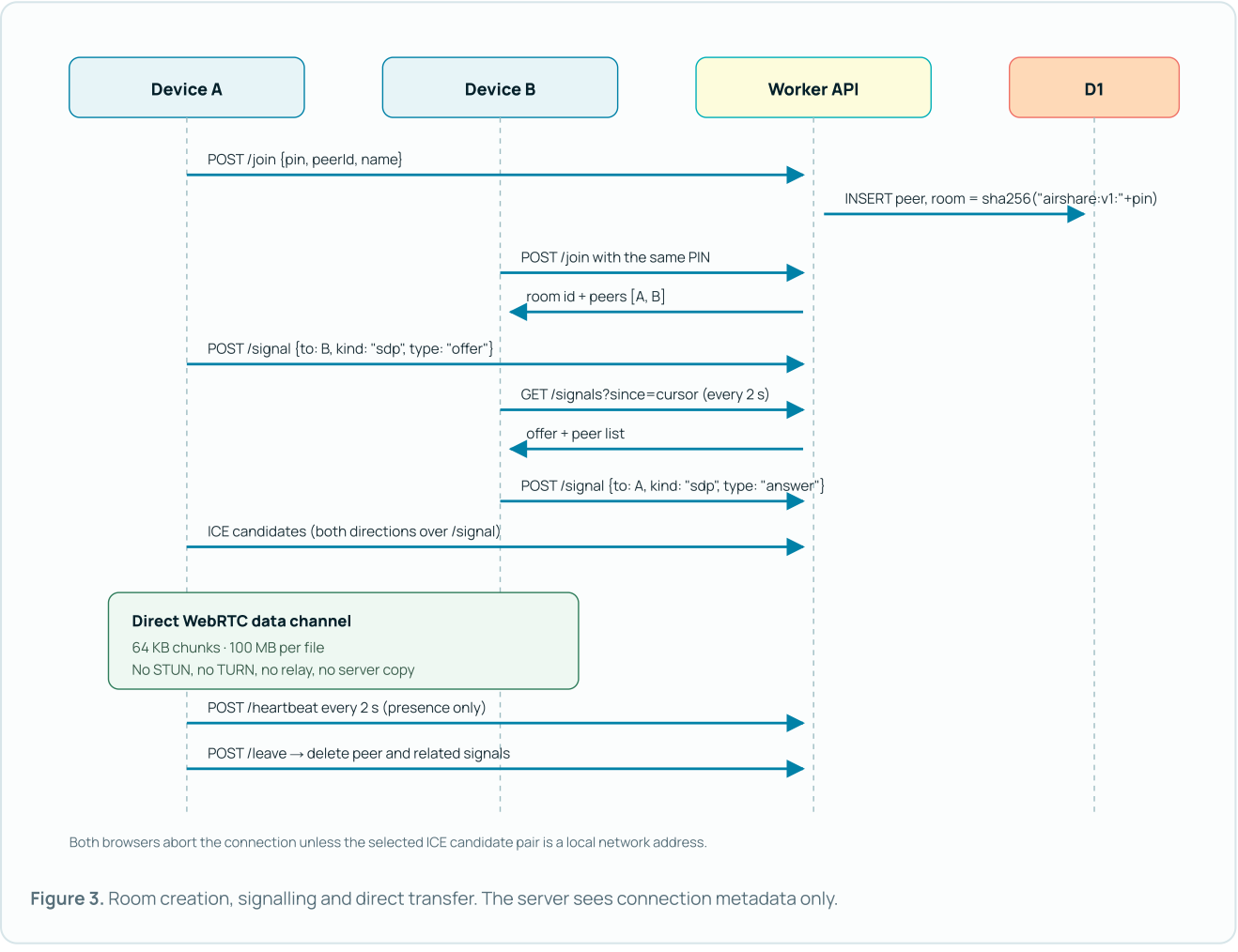


Figure 3. Room creation, signalling and direct transfer. The server sees connection metadata only.

Protocol constants

CONSTANT	VALUE	WHERE ENFORCED
Room id	`sha256("airshare:v1:" + pin)`, 64 hex characters	Worker join
Peer id / session id	8–64 characters, `[A-Za-z0-9_-]`	Join, heartbeat, signals
Device name	1–40 characters, no control characters	Join
Peers per room	8	Join
Peer liveness	90 seconds since `last_seen`	All reads; heartbeat refresh
Signal payload	SDP ≤ 60,000 characters; ICE line ≤ 2,048; total ≤ 64 KB	Signal validation
Signals per poll	50, with an id cursor	`GET /signals`
Signal rate	120 per minute per sending peer	`POST /signal`
Join rate	30 per 10 minutes per IP hash	`POST /join`
File limit	100 MB, 64 KB chunks, whole-file integrity check	Browser client

10

Security design

Transport and headers

HTTPS is terminated by Cloudflare. HSTS is sent with `max-age=31536000; includeSubDomains`. Static responses carry a Content Security Policy that allows only self-hosted scripts, Cloudflare Turnstile and the Cloudflare Web Analytics beacon; inline script and style are permitted because the framework emits them. Frames are limited to Turnstile, and the site cannot be framed by others (`frame-ancestors 'none'`, `X-Frame-Options: DENY`). MIME sniffing is disabled, the referrer is restricted to the origin on cross-origin requests, and camera, microphone and geolocation are disabled by Permissions Policy.

Input handling

- Every API body is parsed defensively; malformed JSON returns 400.
- Content strings are trimmed, length-limited, allow-listed where enumerated, and hrefs are scheme-checked.
- JSON request bodies larger than 1 MB are rejected by Content-Length before parsing. Chunked bodies without a length are not pre-emptively rejected.
- The contact form includes an off-screen honeypot field; filled submissions receive a fake success and are not stored.
- SQL uses bound parameters throughout. Media keys reject `..` and must start with `media/`.

Abuse limits

ACTION	LIMIT	IDENTITY	RESPONSE
Contact message	5 per hour	SHA-256 of IP with “contact:” prefix	429
Admin login	8 per 15 minutes	SHA-256 of IP with “contact:” prefix	429
AirShare join	30 per 10 minutes	SHA-256 of IP with “airshare:” prefix	429
AirShare signal	120 per minute per peer	Peer id within the room	429

Hashes are unsalted, so an attacker who can enumerate likely IP addresses could confirm a match. They are not reversible in practice and are used only for rate limiting.

Secrets

The Worker expects `ADMIN_PASSWORD_HASH`, `SESSION_SECRET`, `TURNSTILE_SECRET` and `GITHUB_TOKEN` as encrypted secrets. Non-secret variables are `SITE_URL` and `GITHUB_REPO`. No secret is present in the repository, the static bundle or the browser. The public Turnstile site key is intentionally visible and has a code fallback.

> **Known exposure surface.** The AirShare room id is a deterministic hash of the PIN. A six-digit PIN space can be enumerated offline if the database ever leaks. The PIN is a room selector, not an authentication secret for stored data; the only data in a room is presence and short-lived signalling, and joins are rate limited online. Do not send files that require confidentiality from the network operator.

11

Performance and caching

- HTML is pre-rendered and served from Cloudflare’s static asset layer; static requests do not invoke the Worker and are not billed as such.
- The build fetches content once; browsers do not call `/api/content` for page rendering.
- Fonts are self-hosted through `next/font` with `display: swap`, so there is no third-party font request.
- Images are local and served as static files; Next image optimization is disabled, which removes a server dependency at the cost of no on-the-fly resizing.
- `/api/content` is cacheable for 60 seconds. R2 media, if enabled, would be immutable for one year. The resume PDF is served with revalidation (`max-age=0, must-revalidate`), confirmed in production.
- AirShare signalling polls every two seconds only while the room is open; it is not used to move file data.

12

SEO, accessibility and browsers

Search metadata

Every page has a title template, a description, a canonical URL and Open Graph/Twitter tags with a 1200 × 630 image. The home page embeds Person JSON-LD with the owner's name, role, college, location, email and social profiles. A sitemap lists the eleven public URLs; `robots.txt` disallows `/admin` and `/api/` and points to the sitemap.

Accessibility

The site provides a skip link, labelled controls, visible focus styles, a keyboard-operable mobile menu, semantic landmarks and reduced reliance on colour alone. The theme toggle is a button with an accessible name. Animations are simple CSS transitions; there is no motion toggle. A formal WCAG audit has not been performed.

Browser requirements

CAPABILITY	REQUIREMENT
Core pages	Any current browser with ES2020 support and CSS custom properties.
Theme memory	`localStorage`; blocked storage falls back to the OS preference.
Contact form	JavaScript enabled; Turnstile widget reachable.
AirShare	WebRTC data channels, `RTCPeerConnection`, async clipboard optional.
Admin	Modern browser with cookie support; no third-party scripts.

13

Known limitations and technical debt

ITEM	STATUS	IMPACT
R2 media storage	NOT ENABLED	No image uploads or resume replacement through the dashboard; resume is served from the repository.
Project images	NOT IMPLEMENTED	Project cards are text-only; the content schema has no image field.
QR code pairing for AirShare	NOT IMPLEMENTED	Users type the PIN manually.
Home leadership cards	HARDCODED	Three curated roles are duplicated in `app/(site)/page.tsx`; editing the dashboard does not change them.
AirShare on restrictive networks	BY DESIGN	No STUN/TURN means some networks cannot connect; the client refuses non-local paths.
AirShare PIN room id	LOW ENTROPY	Room id is a deterministic hash of a short PIN; see the security note.
Admin browser tests	NOT TESTED	No automated signed-in test exists; the owner's password is not available to CI.
Contact Turnstile in headless automation	NOT TESTED	The widget does not mount in headless Chromium; a real public submission was stored on 31 Oct 2026.
Cloudflare zone settings	OWNER ACTION	Always Use HTTPS is observed off and DNSSEC unsigned; both are dashboard settings.

ITEM	STATUS	IMPACT
CSP inline allowances	TRADE-OFF	`'unsafe-inline'` is required by the framework's inline bootstrap and the theme script; a nonce-based CSP is not implemented for a static export.

14

Extending the system

Changes that add a content field require four coordinated edits: the type in `lib/content.ts`, the validator in `worker/content-schema.ts`, the admin field in `components/admin/content-editors.tsx`, and the page that renders it. Adding a new table requires a migration under `migrations/` applied to both local and remote D1. Adding a route is a new folder under `app/(site)/` plus a sitemap entry if it is public. Adding an API route belongs in `worker/index.ts` or `worker/airshare.ts`, and any new origin or script must be added to the CSP in `public/_headers`.

Document 06 provides a step-by-step guide for each of these changes.